Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

Section: A

Instructor Name: Miss Fozia Khan

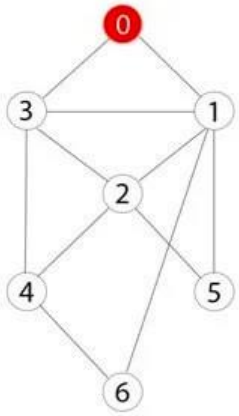
Course: Artificial Intelligence Lab (CS - 325L)

Program: (BS) - Computer Science

Lab #05

Activity- 1:

Consider the following graph. If there is ever a decision between multiple neighbor nodes in the BFS algorithm, assume we always choose the letter closest to the beginning of the alphabet first. A connected graph with 7 nodes and 7 edges. The edges are undirected and unweight. Distance between two nodes will be measured based on the number of edges separating two vertices. Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors.



Define function name _connected_component', this function keep track of all the visited nodes with BFS, is as simple as implementing the steps of the algorithm and assign _queue' variable already has a node to be checked, i.e., the starting vertex that is used as an entry point to explore the graph. The next step is to implement a loop that keeps cycling until queue is empty. At each iteration of the loop, a node is checked. If this wasn't visited already, its neighbors are added to queue. Once the loop is exited, the function (connected_component) returns all of the visited nodes.

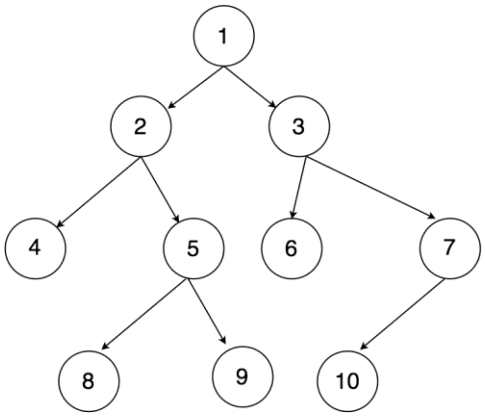
```
from collections import deque

def connected_component(graph, start):
    visited = []
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.append(node)
            # Get neighbors and sort them alphabetically/numerically to ensure correct order
            neighbors = sorted(graph[node])
            for neighbor in neighbors:
                if neighbor not in visited:
                    queue.append(neighbor)
    return visited

# Updated graph based on the image
graph = {
    0: [1, 2, 3],
    1: [0, 5],
    2: [0, 4, 5, 6],
    3: [0, 4],
    4: [2, 3],
    5: [1, 2],
    6: [2]
}

# Starting node for BFS
```

```
start_node = 0

# Get the connected component starting from '0'
component = connected_component(graph, start_node)
print("Connected component starting from", start_node, ":", component)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Connected component starting from 0 : [0, 1, 2, 3, 5, 4, 6]
```

Activity- 2:

Apply Breadth First Search on following graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.




```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
```

```
}

# Initial and goal states
start_node = 1
goal_node = 10

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("Open List:", list(open_list))
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Open List: []
Closed List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Activity- 3:

Repeat Activity-2, apply BFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
}

# Take user input for initial and goal states
start_node = int(input("Enter the initial state: "))
```

```
goal_node = int(input("Enter the goal state: "))

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("Open List:", list(open_list))
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Enter the initial state: 1
Enter the goal state: 5
Open List: [6, 7]
Closed List: [1, 2, 3, 4, 5]
```

Activity- 4:

Apply Depth First Search on Activity-2 graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.

```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found
```

```
# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
}

# Initial and goal states
start_node = 1
goal_node = 10

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
DFS Open List: []
DFS Closed List: [1, 2, 4, 5, 8, 9, 3, 6, 7, 10]
```

Activity- 5:

Repeat Activity-4, apply DFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

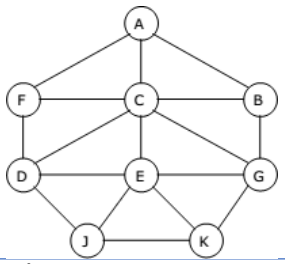
```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)
```

```
    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
}

# Take user input for initial and goal states
start_node = int(input("Enter the initial state: "))
goal_node = int(input("Enter the goal state: "))

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Enter the initial state: 1
Enter the goal state: 6
DFS Open List: [7]
DFS Closed List: [1, 2, 4, 5, 8, 9, 3, 6]
```

Activity- 6:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Breadth First Search on following graph considering the initial state is A and final state is K. Also evaluate it manually.



```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# Initial and goal states
start_node = 'A'
goal_node = 'K'

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("BFS Open List:", list(open_list))
print("BFS Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
BFS Open List: []
BFS Closed List: ['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K']
```

Activity- 7:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Depth First Search on Activity-6 graph considering the initial state is A and final state is K. Also evaluate it manually.

```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
```

```
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# Initial and goal states
start_node = 'A'
goal_node = 'K'

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("BFS Open List:", list(open_list))
```

```
print("BFS Closed List:", closed_list)

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

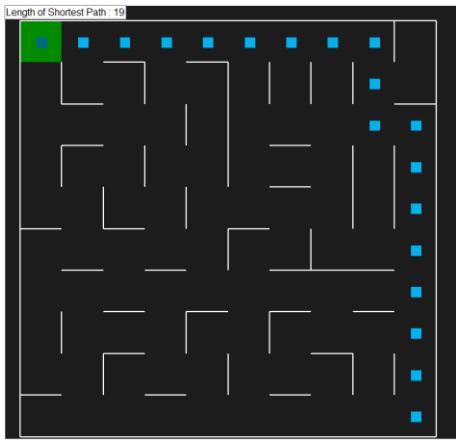
```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
BFS Open List: []
BFS Closed List: ['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K']
DFS Open List: ['F', 'C']
DFS Closed List: ['A', 'B', 'G', 'E', 'J', 'D', 'K']
```

Activity 08:
You need to implement DFS and BFS both using "Pygame" and "Pyamaze" library, generate the maze of (20,20) and find the path.

Implementing BFS using Pyamaze:

```
from pyamaze import maze,agent,textLabel

def BFS(m):
    start=(m.rows,m.cols)
    frontier=[start]
    explored=[start]
    bfsPath={}
    while len(frontier)>0:
        currCell=frontier.pop(0)
        if currCell==(1,1):
            break
        for d in 'ESNW':
            if m.maze_map[currCell][d]==True:
                if d=='E':
                    childCell=(currCell[0],currCell[1]+1)
                elif d=='W':
                    childCell=(currCell[0],currCell[1]-1)
                elif d=='N':
                    childCell=(currCell[0]-1,currCell[1])
                elif d=='S':
                    childCell=(currCell[0]+1,currCell[1])
                if childCell in explored:
                    continue
                frontier.append(childCell)
                explored.append(childCell)
                bfsPath[childCell]=currCell
    fwdPath={}
    cell=(1,1)
    while cell!=start:
        fwdPath[bfsPath[cell]]=cell
        cell=bfsPath[cell]
    return fwdPath

if __name__=='__main__':
    m=maze(10  ,10)
    m.CreateMaze(loopPercent=100)
```

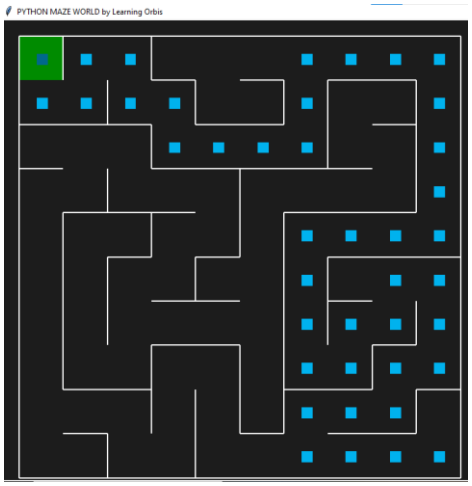
```
path=BFS(m)

a=agent(m,footprints=True)
m.tracePath({a:path})
l=textLabel(m,'Length of Shortest Path',len(path)+1)
print('Length of Shortest Path',len(path)+1)
print(m.maze_map)
m.run()
```



Implementing DFS using Pyamaze:

```
from pyamaze import maze,agent
def DFS(m):
    start=(m.rows,m.cols)
    explored=[start]
    frontier=[start]
    dfsPath={}
    while len(frontier)>0:
        currCell=frontier.pop()
        if currCell==(1,1):
            break
        for d in 'ESNW':
            if m.maze_map[currCell][d]==True:
                if d=='E':
                    childCell=(currCell[0],currCell[1]+1)
                elif d=='W':
                    childCell=(currCell[0],currCell[1]-1)
                elif d=='S':
                    childCell=(currCell[0]+1,currCell[1])
                elif d=='N':
                    childCell=(currCell[0]-1,currCell[1])
                if childCell in explored:
                    continue
                explored.append(childCell)
                frontier.append(childCell)
                dfsPath[childCell]=currCell
    fwdPath={}
    cell=(1,1)
    while cell!=start:
        fwdPath[dfsPath[cell]]=cell
        cell=dfsPath[cell]
    return fwdPath
```

```
if __name__=='__main__':
    m=maze(10 ,10)  #Setting maze 10 X 10
    m.CreateMaze()          # Now creating maze of soze 10 x 10
    path=DFS(m)    #calling method DFS which is returning path
    a=agent(m,footprints=True) #player declaration
    m.tracePath({a:path})       #tracing path by agent

   # print({a:path})
    m.run()   #Execution
```

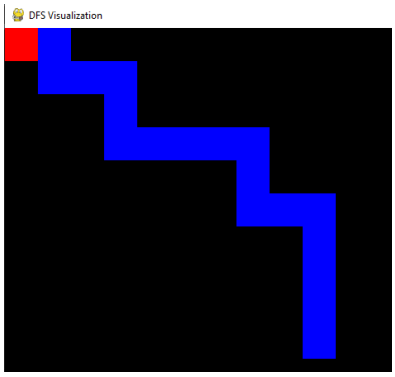


Implementing DFS using Pygame:

```
import pygame
import time

# Initialize Pygame
pygame.init()

# Screen dimensions
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
CELL_SIZE = 40  # Size of each cell in the grid

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
GREEN = (0, 255, 0)

# Create the screen
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("DFS Visualization")

# Maze dimensions
ROWS = 10
COLS = 10

# Create a grid-based maze (1 = open, 0 = wall)
maze = [
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
```

```
    [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
]

# DFS algorithm
def dfs(maze, start, goal):
    stack = [start]
    visited = set()
    path = {}

    while stack:
        current = stack.pop()
        if current == goal:
            break
        visited.add(current)

        for direction in [(0, 1), (1, 0), (0, -1), (-1, 0)]:  # Right, Down, Left, Up
            next_cell = (current[0] + direction[0], current[1] + direction[1])
            if (
                0 <= next_cell[0] < ROWS
                and 0 <= next_cell[1] < COLS
                and next_cell not in visited
                and maze[next_cell[0]][next_cell[1]] == 1
            ):
                stack.append(next_cell)
                path[next_cell] = current

    # Reconstruct the path
    full_path = []
    cell = goal
    while cell != start:
        full_path.append(cell)
        cell = path[cell]
    full_path.append(start)
    full_path.reverse()
    return full_path

# Draw the maze
def draw_maze(maze, path, player_pos):
    for row in range(ROWS):
        for col in range(COLS):
            color = WHITE if maze[row][col] == 1 else BLACK
            pygame.draw.rect(
                screen,
                color,
                (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE),
            )
            pygame.draw.rect(
                screen, BLACK, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1
            )

    # Draw the path
    for cell in path:
```

```
            pygame.draw.rect(
                screen,
                BLUE,
                (cell[1] * CELL_SIZE, cell[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
            )

        # Draw the player
        pygame.draw.rect(
            screen,
            RED,
            (player_pos[1] * CELL_SIZE, player_pos[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
        )

# Main function
def main():
    start = (0, 0)
    goal = (9, 9)
    path = dfs(maze, start, goal)
    player_pos = start

    run = True
    clock = pygame.time.Clock()

    while run:
        screen.fill(BLACK)
        draw_maze(maze, path, player_pos)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_w:  # Move up
                    next_pos = (player_pos[0] - 1, player_pos[1])
                elif event.key == pygame.K_s:  # Move down
                    next_pos = (player_pos[0] + 1, player_pos[1])
                elif event.key == pygame.K_a:  # Move left
                    next_pos = (player_pos[0], player_pos[1] - 1)
                elif event.key == pygame.K_d:  # Move right
                    next_pos = (player_pos[0], player_pos[1] + 1)
                else:
                    next_pos = player_pos

                # Check if the move is valid
                if (
                    0 <= next_pos[0] < ROWS
                    and 0 <= next_pos[1] < COLS
                    and maze[next_pos[0]][next_pos[1]] == 1
                ):
                    player_pos = next_pos

        pygame.display.update()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```



Implementing BFS using Pygame:

```
import pygame
from collections import deque

# Initialize Pygame
pygame.init()

# Screen dimensions
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
CELL_SIZE = 40  # Size of each cell in the grid

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
GREEN = (0, 255, 0)

# Create the screen
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("BFS Visualization")

# Maze dimensions
ROWS = 10
COLS = 10

# Create a grid-based maze (1 = open, 0 = wall)
maze = [
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
]

# BFS algorithm
def bfs(maze, start, goal):
    queue = deque([start])
    visited = set()
```

```
    visited.add(start)
    path = {}

    while queue:
        current = queue.popleft()
        if current == goal:
            break

        for direction in [(0, 1), (1, 0), (0, -1), (-1, 0)]:  # Right, Down, Left, Up
            next_cell = (current[0] + direction[0], current[1] + direction[1])
            if (
                0 <= next_cell[0] < ROWS
                and 0 <= next_cell[1] < COLS
                and next_cell not in visited
                and maze[next_cell[0]][next_cell[1]] == 1
            ):
                queue.append(next_cell)
                visited.add(next_cell)
                path[next_cell] = current

    # Reconstruct the path
    full_path = []
    cell = goal
    while cell != start:
        full_path.append(cell)
        cell = path[cell]
    full_path.append(start)
    full_path.reverse()
    return full_path

# Draw the maze
def draw_maze(maze, path, player_pos):
    for row in range(ROWS):
        for col in range(COLS):
            color = WHITE if maze[row][col] == 1 else BLACK
            pygame.draw.rect(
                screen,
                color,
                (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE),
            )
            pygame.draw.rect(
                screen, BLACK, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1
            )

    # Draw the path
    for cell in path:
        pygame.draw.rect(
            screen,
            BLUE,
            (cell[1] * CELL_SIZE, cell[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
        )

    # Draw the player
    pygame.draw.rect(
        screen,
        RED,
        (player_pos[1] * CELL_SIZE, player_pos[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
    )
```

```
# Main function
def main():
    start = (0, 0)
    goal = (9, 9)
    path = bfs(maze, start, goal)
    player_pos = start

    run = True
    clock = pygame.time.Clock()

    while run:
        screen.fill(BLACK)
        draw_maze(maze, path, player_pos)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_w:  # Move up
                    next_pos = (player_pos[0] - 1, player_pos[1])
                elif event.key == pygame.K_s:  # Move down
                    next_pos = (player_pos[0] + 1, player_pos[1])
                elif event.key == pygame.K_a:  # Move left
                    next_pos = (player_pos[0], player_pos[1] - 1)
                elif event.key == pygame.K_d:  # Move right
                    next_pos = (player_pos[0], player_pos[1] + 1)
                else:
                    next_pos = player_pos

                # Check if the move is valid
                if (
                    0 <= next_pos[0] < ROWS
                    and 0 <= next_pos[1] < COLS
                    and maze[next_pos[0]][next_pos[1]] == 1
                ):
                    player_pos = next_pos

        pygame.display.update()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```